TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 2: Meet the OS You Will Build

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Abbreviations **4**

Introduction **5**

1 Build environment and boot **6**

1.1 Source tree and toolchain 6

1.2 Booting in QEMU 6

2 Using xv6 as a Unix system **7**

2.1 Included programs and file contents 7

2.2 Pipelines and shell comparison 8

3 Reading userspace and kernel source **9**

3.1 User programs and system calls 9

3.2 Kernel handlers and privilege separation 11

4 Implementing and testing sleep **13**

4.1 Program source and version adaptation 13

4.2 Build registration 14

4.3 Observed runtime results 15

Conclusions **17**

ABSTRACT

This laboratory explores xv6 through compilation, booting, shell commands, source inspection, and the addition of a user program. The operating system was run on emulated RISC-V hardware using QEMU from an Ubuntu virtual machine. Existing commands demonstrated directory listing, file reading, output, and pipelines. Source inspection connected a user program's read and write requests with their kernel handlers. A new program named `sleep` was registered in the build and used the `pause` system call declared by the downloaded checkout. Runtime screenshots show a valid call returning to the shell and usage messages for missing and extra arguments. The report distinguishes these observed results from unmeasured properties such as pause duration and exit status.

Keywords: xv6, system calls, user programs, kernel, pipes, emulation

ABBREVIATIONS

OS	operating system
VM	virtual machine
PID	process identifier
CPU	central processing unit
QEMU	the emulator used to run xv6
RISC-V	the instruction-set architecture targeted by this xv6 checkout

INTRODUCTION

This laboratory follows the course exercise on building and using a small teaching operating system, xv6 [**?**]. It complements the external inspection of Linux in Lab 1 by exposing both the programs that request OS services and the kernel code that handles those requests.

The objectives were to boot xv6, use its shell, distinguish userspace from kernel code, locate system-call handlers, and add a working user program. Commands were executed in the Ubuntu terminal and inside the xv6 shell. The figures are the student's captured outputs, and the program listing matches the source displayed in the VM.

1 BUILD ENVIRONMENT AND BOOT

The xv6 source tree was obtained in `~/xv6-riscv`. The build environment uses a RISC-V cross-compiler to generate code for the emulated machine.

1.1 Source tree and toolchain

The directory listing contains `kernel`, `user`, `mkfs`, `Makefile`, and `README`. The tool check located `/usr/bin/riscv64-linux-gnu-gcc`, while the alternative `riscv64-unknown-elf-gcc` check produced no path. QEMU reported version 10.2.1. The header `user/user.h` declared `int pause(int);` at line 25, establishing the system-call name required by this checkout. Figure 1.1 records the checks.

```
victor@oscourse:~/xv6-riscv$ ls
LICENSE  Makefile  README  fs.img  kernel  mkfs  test-xv6.py  user
victor@oscourse:~/xv6-riscv$ command -v riscv64-linux-gnu-gcc
/usr/bin/riscv64-linux-gnu-gcc
victor@oscourse:~/xv6-riscv$ command -v riscv64-unknown-elf-gcc
victor@oscourse:~/xv6-riscv$ qemu-system-riscv64 --version | head -1
QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
victor@oscourse:~/xv6-riscv$ grep -nE 'sleep|pause' user/user.h
25:int pause(int);
victor@oscourse:~/xv6-riscv$
```

Figure 1.1 – Source tree, compiler path, QEMU version, and pause declaration

1.2 Booting in QEMU

The command `make qemu` launched the kernel. The console printed `xv6 kernel is booting`, messages from additional harts, and `init: starting sh` before displaying the shell prompt. A hart is a RISC-V hardware thread. Figure 1.2 demonstrates successful booting. This capture was made after the initial build, so it shows the launch command rather than a fresh compilation transcript.

```
victor@oscourse:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img
,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

Figure 1.2 – xv6 boot messages and interactive shell

2 USING XV6 AS A UNIX SYSTEM

The shell provides a small set of familiar Unix programs. Their outputs demonstrate basic file operations and communication between processes.

2.1 Included programs and file contents

Three supplied programs are `ls`, which lists directory entries; `cat`, which reads file contents; and `echo`, which prints its arguments. The directory listing also includes `grep`, `wc`, `mkdir`, and `sh`. Figure 2.1 shows the listing and the opening part of the README. The README output is an excerpt, whereas the separate word-count command operates on the entire file.

```
hart 1 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
console        3 23 0
$ cat RE
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.

ACKNOWLEDGMENTS

xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
to Peer Communications; ISBN: 1-57398-013-7; 1st edition (June 14,
2000)).  See also https://pdos.csail.mit.edu/6.1810/, which provides
pointers to on-line resources for v6.
```

Figure 2.1 – Included xv6 programs and an excerpt of README

2.2 Pipelines and shell comparison

The command `echo hello xv6` printed `hello xv6`. The pipeline `ls | grep c` produced entries for `cat`, `echo`, `wc`, `sync`, and `console`. The command `wc README` printed `48 336 2441 README`, representing 48 lines, 336 words, and 2441 bytes. Figure 2.2 records these results.

A pipeline needs process creation and execution, together with interprocess communication. Separate processes can be started with `fork` and `exec`; a kernel pipe transports bytes through file descriptors from one program's standard output to another's standard input. The xv6 shell supports familiar execution and pipes, but provides fewer conveniences than the Linux shell used in Lab 1.

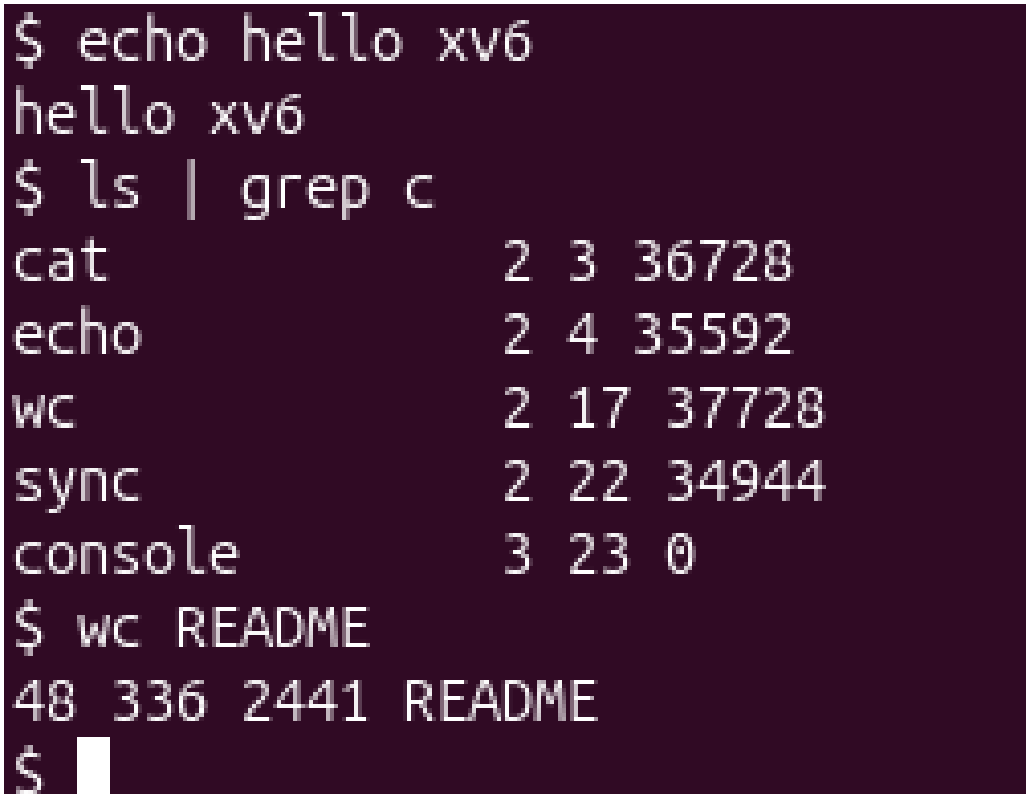


Figure 2.2 – Output, pipeline filtering, and README counts

3 READING USERSPACE AND KERNEL SOURCE

The source tree separates ordinary programs in `user/` from privileged OS code in `kernel/`. This distinction makes it possible to trace a request from a user program to its kernel implementation.

3.1 User programs and system calls

The first 40 lines of `user/ls.c` show its includes, filename-formatting helper, and the opening of a path. The complete `user/cat.c` listing shows how bytes are read into a buffer and written to standard output. Figures 3.1 and 3.2 record the inspected code.

```
victor@oscourse:~/xv6-riscv$ sed -n '1,40p' user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
  buf[sizeof(buf) - 1] = '\0';
  return buf;
}

void
ls(char *path)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "ls: cannot open %s\n", path);
    return;
  }
```

Figure 3.1 – Beginning of the xv6 directory-listing program

```
victor@oscourse:~/xv6-riscv$ cat -n user/cat.c
     1  #include "kernel/types.h"
     2  #include "kernel/fcntl.h"
     3  #include "user/user.h"
     4
     5  char buf[512];
     6
     7  void
     8  cat(int fd)
     9  {
    10    int n;
    11
    12    while ((n = read(fd, buf, sizeof(buf))) > 0) {
    13      if (write(1, buf, n) != n) {
    14        fprintf(2, "cat: write error\n");
    15        exit(1);
    16      }
    17    }
    18    if (n < 0) {
    19      fprintf(2, "cat: read error\n");
    20      exit(1);
    21    }
    22  }
    23
    24  int
    25  main(int argc, char *argv[])
    26  {
    27    int fd, i;
    28
    29    if (argc <= 1) {
    30      cat(0);
    31      exit(0);
    32    }
    33
    34    for (i = 1; i < argc; i++) {
    35      if ((fd = open(argv[i], O_RDONLY)) < 0) {
    36        fprintf(2, "cat: cannot open %s\n", argv[i]);
    37        exit(1);
    38      }
    39      cat(fd);
    40      close(fd);
    41    }
    42    exit(0);
    43  }
victor@oscourse:~/xv6-riscv$
```

Figure 3.2 – Complete source of the xv6 cat program

Table 3.1 identifies the direct system calls visible in `cat.c`. With no file arguments, the program reads standard input, descriptor 0. Its output goes to descriptor 1.

Table 3.1 – System calls used by the cat program

Call	**Request to the kernel**
`open`	Open an input pathname and obtain a descriptor.
`read`	Transfer input bytes into the userspace buffer.
`write`	Transfer bytes from the buffer to standard output.
`close`	Release the opened descriptor.
`exit`	Terminate the process with a success or error status.

The function `fprintf` formats output in userspace; it is not itself a system call, although its output ultimately uses `write`. Figure 3.3 shows the system-call declarations followed by userspace library declarations in `user/user.h`.

```
victor@oscourse:~/xv6-riscv$ cat -n user/user.h
     1  #define SBRK_ERROR ((char *)-1)
     2
     3  struct stat;
     4
     5  // system calls
     6  int fork(void);
     7  int exit(int) __attribute__((noreturn));
     8  int wait(int *);
     9  int pipe(int *);
    10  int write(int, const void *, int);
    11  int read(int, void *, int);
    12  int close(int);
    13  int kill(int);
    14  int exec(const char *, char **);
    15  int open(const char *, int);
    16  int mknod(const char *, short, short);
    17  int unlink(const char *);
    18  int fstat(int fd, struct stat *);
    19  int link(const char *, const char *);
    20  int mkdir(const char *);
    21  int chdir(const char *);
    22  int dup(int);
    23  int getpid(void);
    24  char *sys_sbrk(int, int);
    25  int pause(int);
    26  int uptime(void);
    27  int sync(void);
    28
    29  // ulib.c
    30  int stat(const char *, struct stat *);
    31  char *strcpy(char *, const char *);
    32  void *memmove(void *, const void *, int);
    33  char *strchr(const char *, char c);
    34  int strcmp(const char *, const char *);
    35  char *gets(char *, int max);
    36  uint strlen(const char *);
    37  void *memset(void *, int, uint);
    38  int atoi(const char *);
    39  int memcmp(const void *, const void *, uint);
    40  void *memcpy(void *, const void *, uint);
    41  char *sbrk(int);
    42  char *sbrklazy(int);
    43
    44  // printf.c
    45  void fprintf(int, const char *, ...) __attribute__((format(printf, 2, 3)));
    46  void printf(const char *, ...) __attribute__((format(printf, 1, 2)));
    47
    48  // umalloc.c
    49  void *malloc(uint);
    50  void free(void *);
victor@oscourse:~/xv6-riscv$
```

Figure 3.3 – System-call interface and userspace library declarations

3.2 Kernel handlers and privilege separation

In the inspected checkout, `sys_read(void)` is at line 69 of `kernel/sysfile.c`; its return type appears on line 68. The handler retrieves arguments, checks the file descriptor, and calls `fileread`. The handler `sys_write(void)` is at line 83 and calls `filewrite`. Figure 3.4 records both implementations. These line numbers refer to the captured source version.

The code in `kernel/` runs with privilege to manage resources and implement OS services, while the code in `user/` runs as ordinary programs and requests those services through system calls.

```
victor@oscourse:~/xv6-riscv$
victor@oscourse:~/xv6-riscv$ grep -n -A 12 -B 2 -E '^sys_(read|write)\(' kernel/sysfile.c
67-
68-uint64
69:sys_read(void)
70-{
71-  struct file *f;
72-  int n;
73-  uint64 p;
74-
75-  argaddr(1, &p);
76-  argint(2, &n);
77-  if (argfd(0, 0, &f) < 0)
78-    return -1;
79-  return fileread(f, p, n);
80-}
81-
82-uint64
83:sys_write(void)
84-{
85-  struct file *f;
86-  int n;
87-  uint64 p;
88-
89-  argaddr(1, &p);
90-  argint(2, &n);
91-  if (argfd(0, 0, &f) < 0)
92-    return -1;
93-
94-  return filewrite(f, p, n);
95-}
victor@oscourse:~/xv6-riscv$
```

Figure 3.4 – Kernel implementations of read and write

4 IMPLEMENTING AND TESTING SLEEP

The new program pauses for a requested number of clock ticks and returns to the shell. Its executable name is `sleep`, while its kernel request uses the `pause` interface declared by this checkout.

4.1 Program source and version adaptation

The handout's example calls `sleep(atoi(argv[1]))`. The inspected `user/user.h` declares `pause` instead, so the implementation calls `pause(atoi(argv[1]))`. This adapts the system-call name while preserving the executable and usage syntax required by the exercise.

The program requires exactly one command-line argument. Otherwise it prints `usage: sleep <ticks>` to standard error and calls `exit(1)`. For a valid argument count, `atoi` converts the argument to an integer, `pause` requests the delay, and `exit(0)` terminates the program. The source is shown below and in Figure 4.1.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  pause(atoi(argv[1]));
  exit(0);
}
```

```
victor@oscourse:~/xv6-riscv$ cat > user/sleep.c <<'EOF'
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  pause(atoi(argv[1]));
  exit(0);
}
EOF
cat -n user/sleep.c
     1  #include "kernel/types.h"
     2  #include "user/user.h"
     3
     4  int
     5  main(int argc, char *argv[])
     6  {
     7    if(argc != 2){
     8      fprintf(2, "usage: sleep <ticks>\n");
     9      exit(1);
    10    }
    11
    12    pause(atoi(argv[1]));
    13    exit(0);
    14  }
victor@oscourse:~/xv6-riscv$
```

Figure 4.1 – Creating and displaying the sleep source in the VM

4.2 Build registration

The executable was added to the `UPROGS` list as follows:

```
  $U/_sync\
  $U/_sleep\

fs.img: mkfs/mkfs README $(UPROGS)
```

The actual Makefile entries use tab indentation. A blank line is preserved before `fs.img:` because the final list entry ends with a continuation backslash. Figure 4.2 records an earlier edit in which this separator had been removed; that capture is not a representation of the correct final layout. The repository includes `Makefile.patch` with the intended registration and separator.

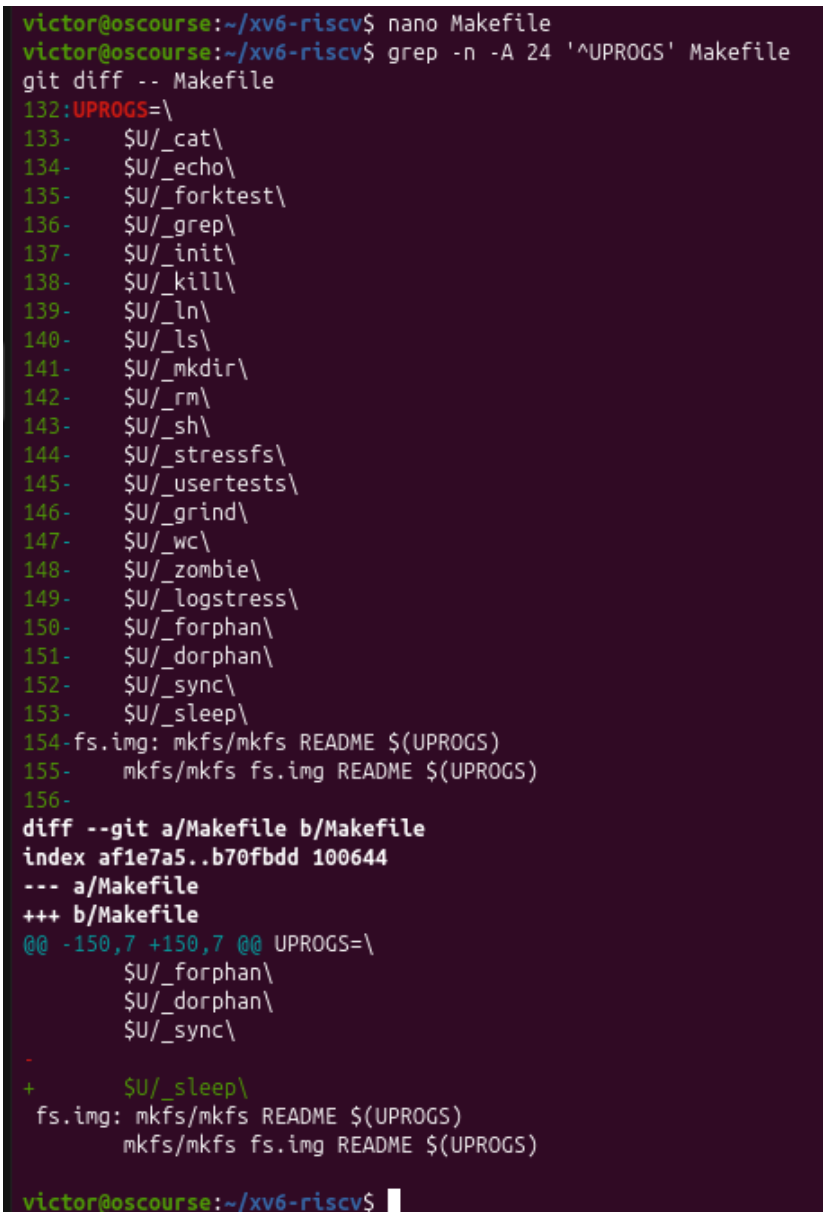


Figure 4.2 – Earlier Makefile edit before restoring the blank separator

4.3 Observed runtime results

The runtime sequence printed `before`, ran `sleep 10`, and then printed `after`. The valid call returned to the shell without an error message. Running `sleep` without arguments printed the usage message. Figure 4.3 shows these results and an earlier failed extra-argument attempt.

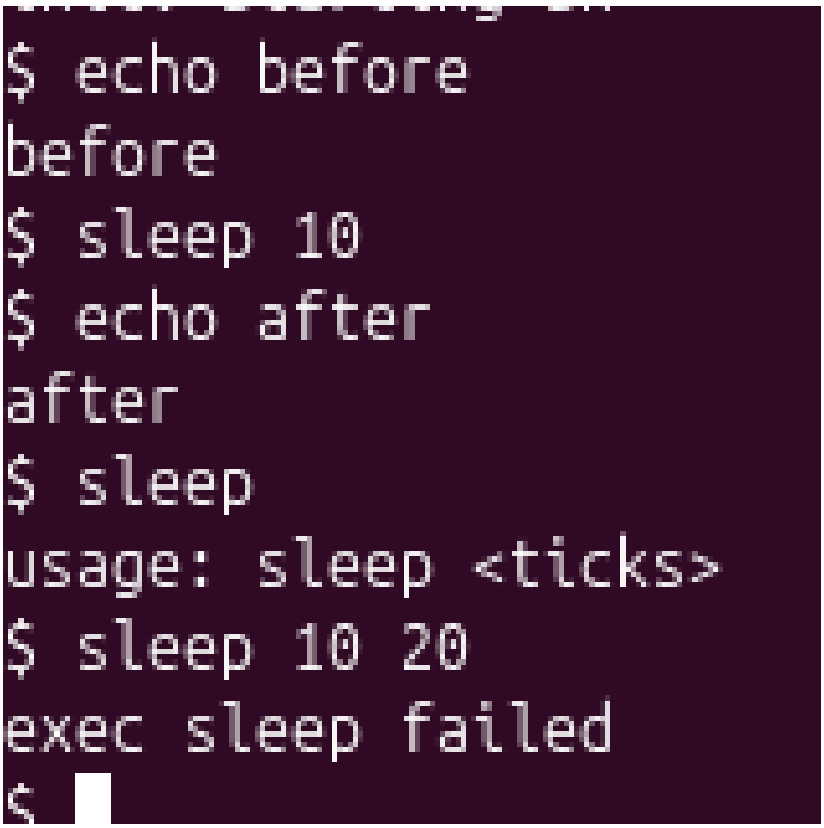


Figure 4.3 – Valid call, missing-argument check, and earlier failed attempt

The repeated command `sleep 10 20` printed `usage: sleep <ticks>`, as shown in Figure 4.4. Both invalid argument counts therefore produced the expected message. The earlier `exec sleep failed` result was superseded by this successful repeat; its cause was not established by the captures.

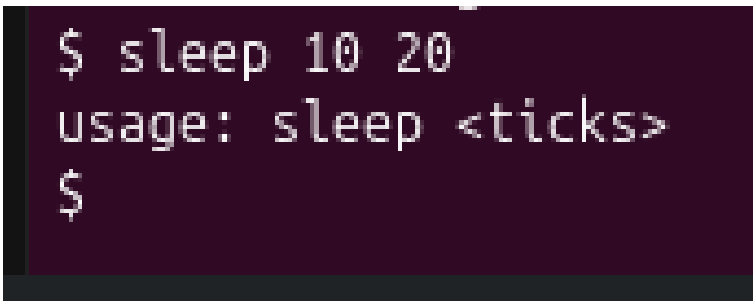


Figure 4.4 – Successful extra-argument usage check

The screenshots show execution and argument-count handling, but do not measure the pause duration or expose exit statuses. No independent rebuild was performed from this report repository because it contains the user program and registration patch rather than the full xv6 source tree.

CONCLUSIONS

xv6 booted successfully in QEMU and provided working file-reading, output, and pipeline commands. Inspecting both `user/cat.c` and `kernel/sysfile.c` connected the user-level read and write requests with their privileged kernel handlers.

The added `sleep` program uses the checkout's `pause` system call and is accompanied by a build-registration patch. The captured runtime results show a valid call returning and the expected usage messages for missing and extra arguments. The repository containing the reports, screenshots, and program is `https://github.com/yoda-dinmd/os-labs`.